Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

VARALAKSHMI **1BM24EC242**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 _ 2027

INDEX:

CONTENTS	PAGE NO
INTRODUCTION OF TOOL	1
ALU- ARITHMETIC LOGIC UNIT	**2**
COMPARATOR	4
4 BIT PARITY GENERATOR AND CHECKER	6
4 BIT BARREL SHIFTER	8
TIMER MODULE	10
RING COUNTER	12
JOHNSON COUNTER	14
SYNCHRONOUS COUNTER	16
ASYNCHRONOUS COUNTER	18
SHIFT REGISTER	20
DIVIDE BY N COUNTER	22
MEALY FSM	24
MOORE FSM	26
RANDOM SEQUENCE GENERATOR	28
CONCLUSION	30

TOOL USED: **EDA PLAYGROUND**

EDA Playground is an online platform used for designing, simulating, and testing digital electronic circuits using Hardware Description Languages (HDLs) such as Verilog, SystemVerilog, VHDL, and other supported languages. EDA stands for Electronic Design Automation. The platform allows students, engineers, and researchers to write and execute HDL code directly in a web browser without installing any additional software. It is especially useful for learning digital electronics, verifying circuit designs, and understanding the behaviour of digital systems.

One of the main features of EDA Playground is its user-friendly interface, which provides separate code editors for the Design Under Test (DUT) and the testbench. The DUT contains the actual hardware design, while the testbench provides input signals and checks the outputs to verify whether the design works correctly. Users can select different simulators and supported HDL languages according to their requirements. After writing the code, they can run the simulation and observe the results in the console. Waveform viewing is also supported through compatible waveform tools, such as EPWave, which helps users analyse signal transitions and understand the timing behaviour of digital circuits.

EDA Playground supports the simulation of various digital circuits, including logic gates, multiplexers, encoders, decoders, adders, counters, flip-flops, shift registers, finite state machines, and sequence detectors. It is widely used in academic laboratories and digital design courses because it provides a convenient environment for testing and debugging HDL programs. For example, a student can write Verilog code for an AND gate, create a testbench with all possible input combinations, execute the simulation, and verify the output without building the physical circuit.

Another important advantage of EDA Playground is that it is accessible from anywhere with an internet connection and a compatible web browser. It reduces the need for expensive development tools and helps students share their code and simulation results easily. However, it is primarily intended for HDL-based simulation and verification; it does not replace every feature of professional electronic design tools used for complete FPGA implementation or physical chip fabrication.

1. ALU – ARITHMETIC LOGIC UNIT

WORKING:

An Arithmetic Logic Unit performs arithmetic and bitwise operations on binary operands. The two 4-bit inputs are processed according to a 3-bit operation selector. A multiplexer-like case statement selects the requested operation, such as addition, subtraction, AND, OR, XOR, NOT, left shift or right shift. The result is provided at the output along with carry and zero status signals. The testbench applies different operation codes and checks the resulting outputs.

VERILOG DESIGN CODE:

```
// Code your testbench here
// or browse Examples
`timescale 1ns/1ps
module tb;
reg [3:0] a,b;
reg [2:0] op;
wire [3:0] y;
wire carry,zero;

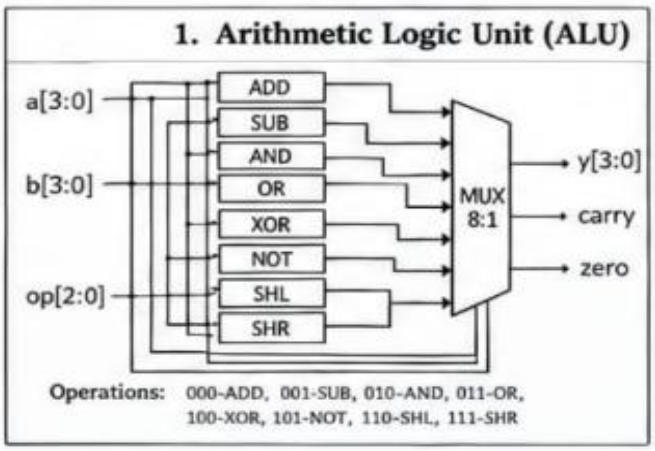
alu4 dut(a,b,op,y,carry,zero);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);
    a=7; b=3;
    op=0; #10;
    op=1; #10;
    op=2; #10;
    op=3; #10;
    op=4; #10;
    op=5; #10;
    op=6; #10;
    op=7; #10;
    $finish;
end
endmodule
```

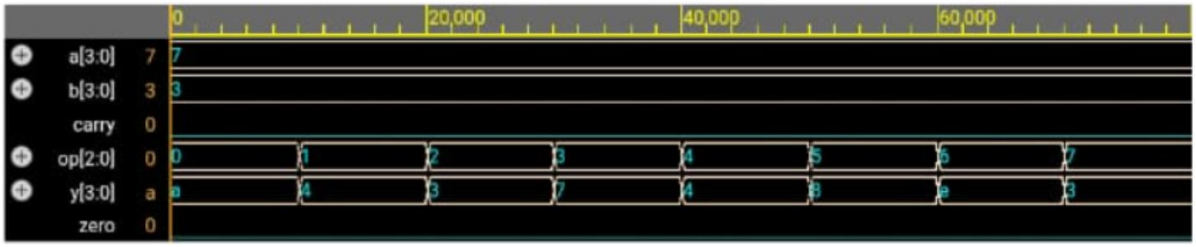
TESTBENCH:

```
// Code your design here
module alu4(
    input [3:0] a, b,
    input [2:0] op,
    output reg [3:0] y,
    output reg carry, zero
);
always @(*) begin
    carry = 0;
    case(op)
        3'b000: {carry,y} = {1'b0,a} + {1'b0,b};
        3'b001: y = a - b;
        3'b010: y = a & b;
        3'b011: y = a | b;
        3'b100: y = a ^ b;
        3'b101: y = ~a;
        3'b110: y = a << 1;
        3'b111: y = a >> 1;
    endcase
    zero = (y == 0);
end
endmodule
```

CIRCUIT DESIGN:



OUTPUT WAVEFORM:



2.COMPARATOR

WORKING:

A digital comparator evaluates two binary numbers and determines their relationship. The circuit compares the corresponding bits of the two 4-bit inputs. It produces separate outputs indicating whether A is greater than B, equal to B, or less than B. Equality is determined when all corresponding bits match, while magnitude outputs reflect the binary comparison. The testbench applies input pairs to demonstrate all comparison conditions.

VERILOG DESIGN CODE:

```
// Code your design here
module comparator4(
    input [3:0] a,b,
    output eq,gt,lt
);
assign eq = (a == b);
assign gt = (a > b);
assign lt = (a < b);
endmodule
```

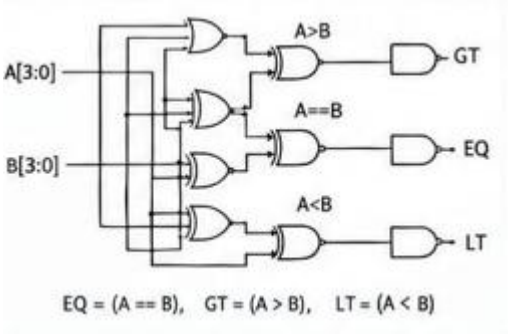
TESTBENCH:

```
// Code your testbench here
// or browse Examples
`timescale 1ns/1ps
module tb;
reg [3:0] a,b;
wire eq,gt,lt;

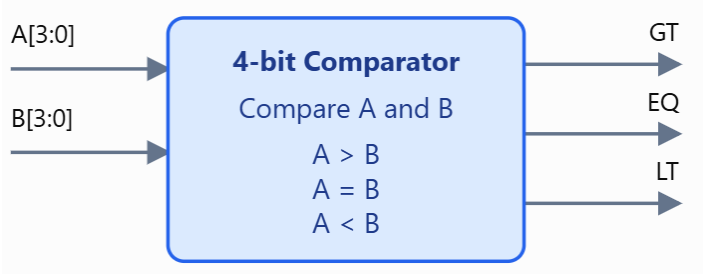
comparator4 dut(a,b,eq,gt,lt);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);
    a=5; b=5; #10;
    a=9; b=3; #10;
    a=2; b=7; #10;
    $finish;
end
endmodule
```

CIRCUIT DESIGN:



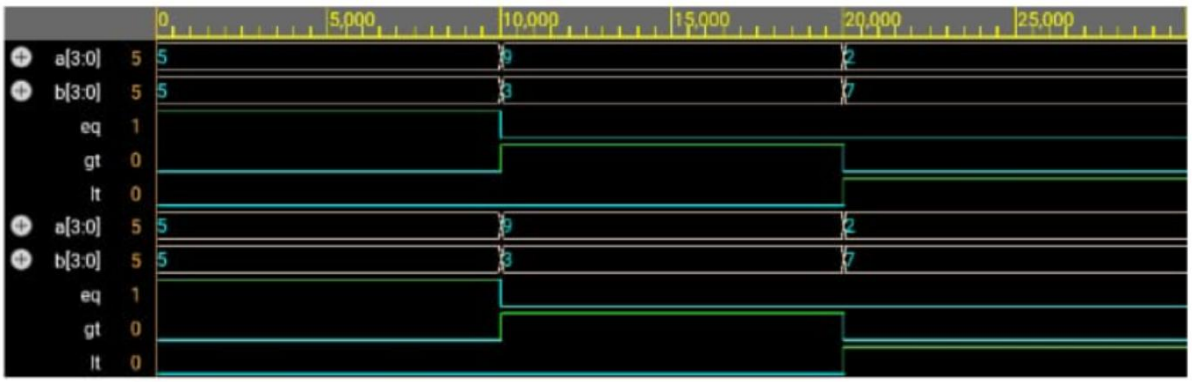
BLOCK DIAGRAM:



EXAMPLE:

A	B	GT	EQ	LT
5	3	1	0	0
3	3	0	1	0
2	7	0	0	1

OUTPUT WAVEFORM:



3. 4-bit Parity Generator and Checker

WORKING:

A parity circuit adds a parity bit to data or checks parity in received data. XOR operations combine the four data bits to calculate the parity. Even parity makes the total number of logic-one bits even, while odd parity is the complement of the even-parity value. The checker compares the calculated parity with the received parity bit. A parity-error output indicates a mismatch and can reveal some single-bit transmission errors.

VERILOG CODE:

```
// Code your design here
module parity4(
    input [3:0] data,
    input
received_even_parity,
    output even_parity,
    output odd_parity,
    output parity_error
);
assign even_parity =
^data;
assign odd_parity = ~
(^data);
assign parity_error =
    (^data) ^
received_even_parity;
endmodule
```

TESTBENCH:

```
// Code your testbench
here
// or browse Examples
`timescale 1ns/1ps
module tb;
reg [3:0] data;
reg received_even_parity;
wire
even_parity,odd_parity,par
ity_error;

parity4
dut(data,received_even_par
ity,

even_parity,odd_parity,par
ity_error);

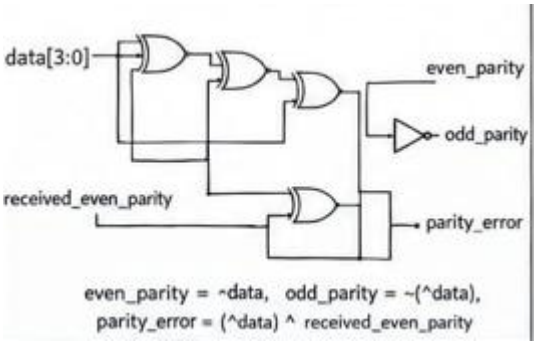
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);
    data=4'b1010;

received_even_parity=1;
#10;
    data=4'b1111;

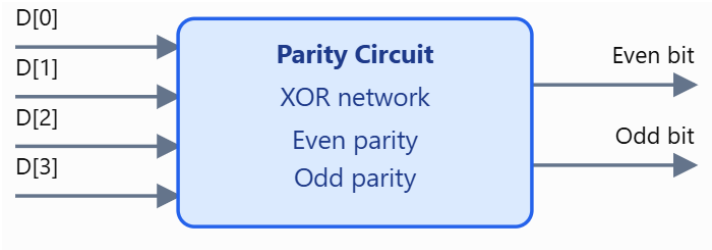
received_even_parity=0;
#10;
    data=4'b0001;

received_even_parity=1;
#10;
    $finish;
end
endmodule
```

CIRCUIT DESIGN:



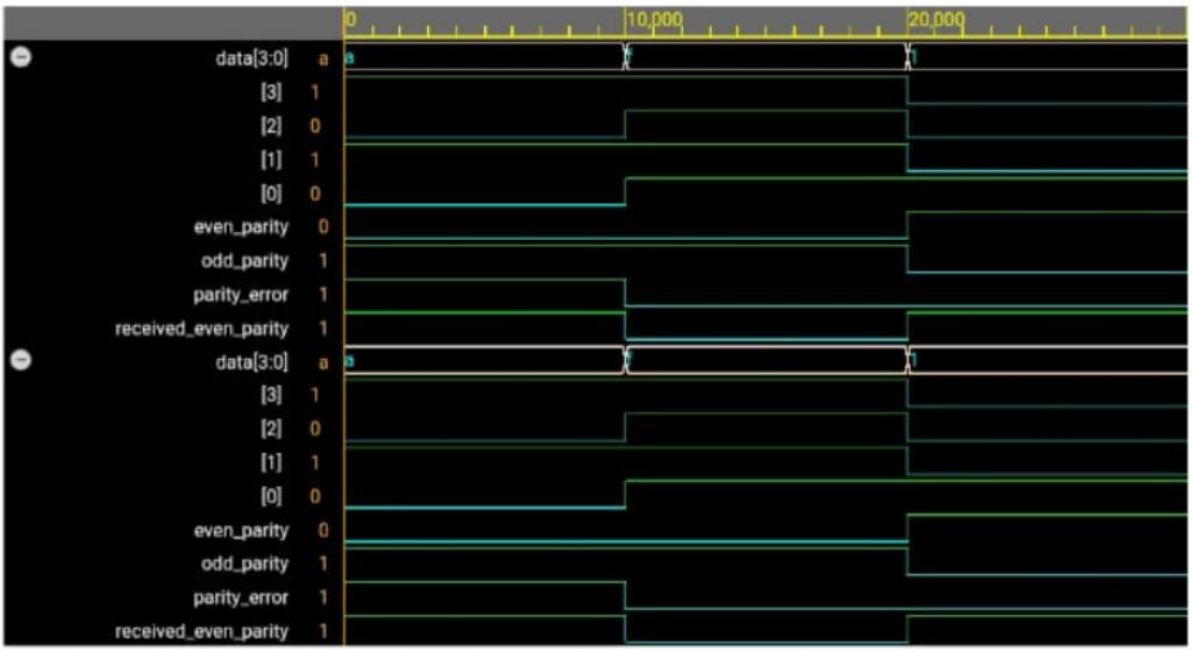
BLOCK DIAGRAM:



EXAMPLE:

Data	Received even parity	Generated even parity	Odd parity	Error
1011	1	1	0	0
1011	0	1	0	1
1111	0	0	1	0
1111	1	0	1	1

OUTPUT WAVEFORM:



4. 4-bit Barrel Shifter

WORKING:

A barrel shifter shifts a multi-bit word by a selected number of positions in one combinational operation. The 4-bit input is routed through stages of multiplexers controlled by the shift amount. A direction signal chooses between left and right shifting. The output reflects the shifted data according to the selected amount and direction. The testbench changes the amount and direction to verify the supported shift cases.

VERILOG CODE:

```
// Code your design here
module barrel_shifter4(
    input [3:0] data,
    input [1:0] amount,
    input dir,
    output reg [3:0] y
);
always @(*) begin
    if (dir == 0)
        y = data <<
amount;
    else
        y = data >>
amount;
end
endmodule
```

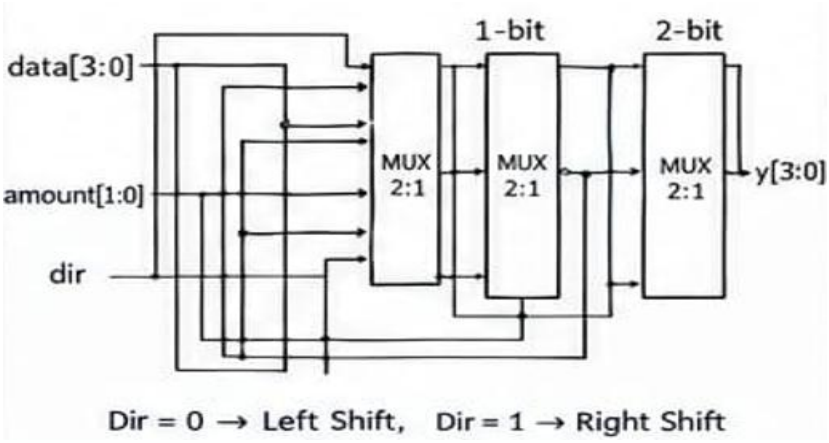
TESTBENCH:

```
// Code your testbench
here
// or browse Examples
`timescale 1ns/1ps
module tb;
reg [3:0] data;
reg [1:0] amount;
reg dir;
wire [3:0] y;

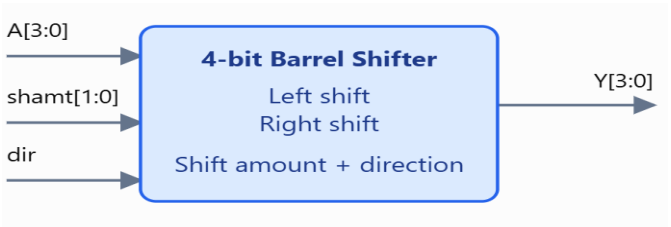
barrel_shifter4
dut(data,amount,dir,y);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);
    data=4'b1011;
    dir=0; amount=1; #10;
    amount=2; #10;
    dir=1; amount=1; #10;
    amount=2; #10;
    $finish;
end
endmodule
```

CIRCUIT DESIGN:



BLOCK DIAGRAM:

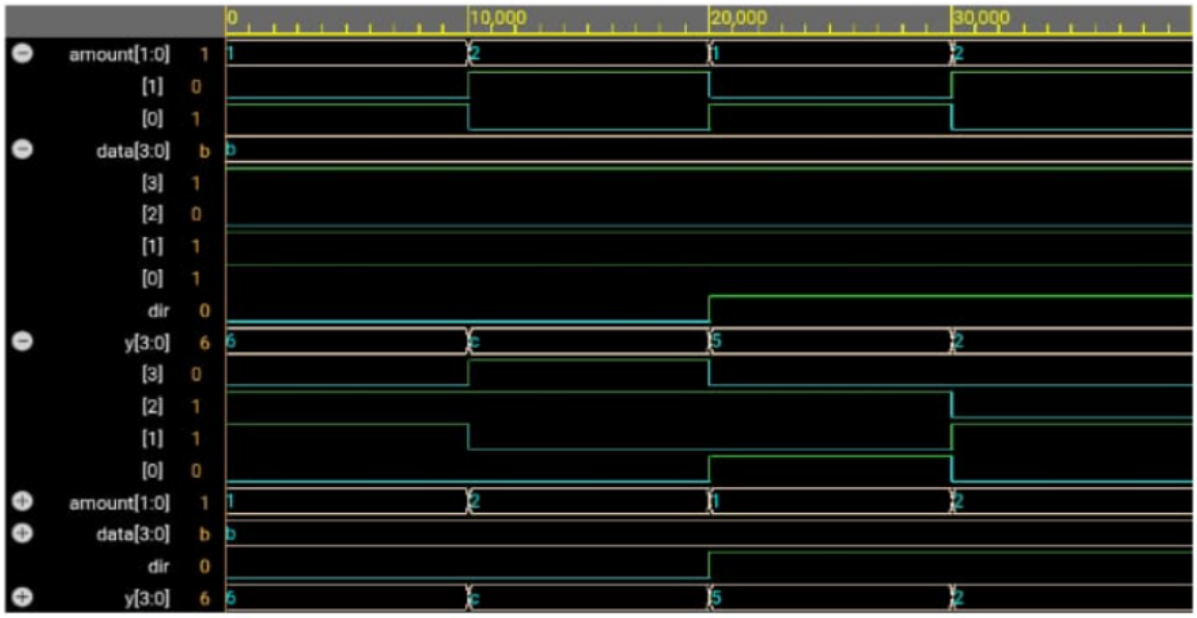


EXAMPLE:

A	dir	Shift amount	Y
1011	0	1	0110
1011	0	2	1100
1011	1	1	0101
1011	1	2	0010

Here, `dir = 0` means left shift and `dir = 1` means right shift.

OUTPUT WAVEFORM:



5. Timer Module

WORKING:

A timer module counts clock events and generates a pulse when a programmed count is reached. The clock drives the sequential counter elements, while reset initializes the state and enable controls counting. The count advances on the active clock edge until the terminal count is detected. At the specified count, the circuit asserts a tick output and then continues or restarts according to the design logic. The testbench verifies reset, enable, counting and tick generation.

VERILOG CODE:

```
// Code your design here
module timer_module(
    input clk, reset,
enable,
    output reg tick
);
reg [3:0] count;

always @(posedge clk or
posedge reset) begin
    if (reset) begin
        count <= 0;
        tick <= 0;
    end
    else if (enable)
begin
        if (count == 4)
begin
            count <= 0;
            tick <= 1;
        end
        else begin
            count <=
count + 1;
            tick <= 0;
        end
    end
    else tick <= 0;
end
endmodule
```

TESTBENCH:

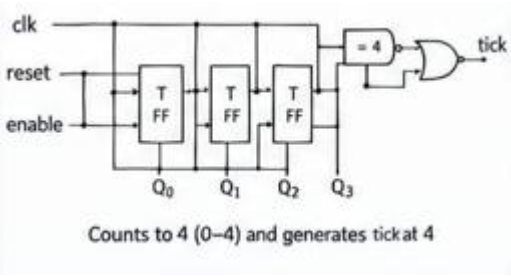
```
// Code your testbench
here
// or browse Examples
`timescale 1ns/1ps
module tb;
reg clk=0, reset=1,
enable=1;
wire tick;

always #5 clk=~clk;

timer_module
dut(clk,reset,enable,tick)
;

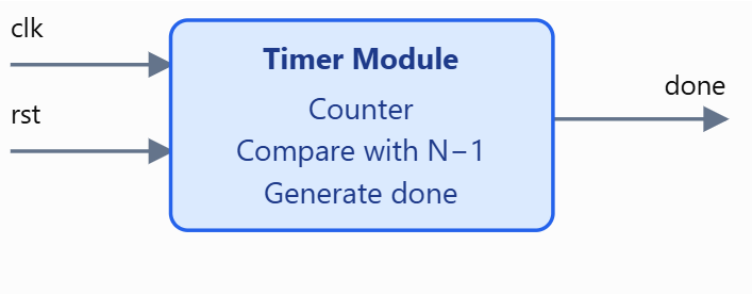
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);
    #12 reset=0;
    #120;
    $finish;
end
endmodule
```

CIRCUIT DESIGN :



Counts to 4 (0–4) and generates tick at 4

BLOCK DIAGRAM:

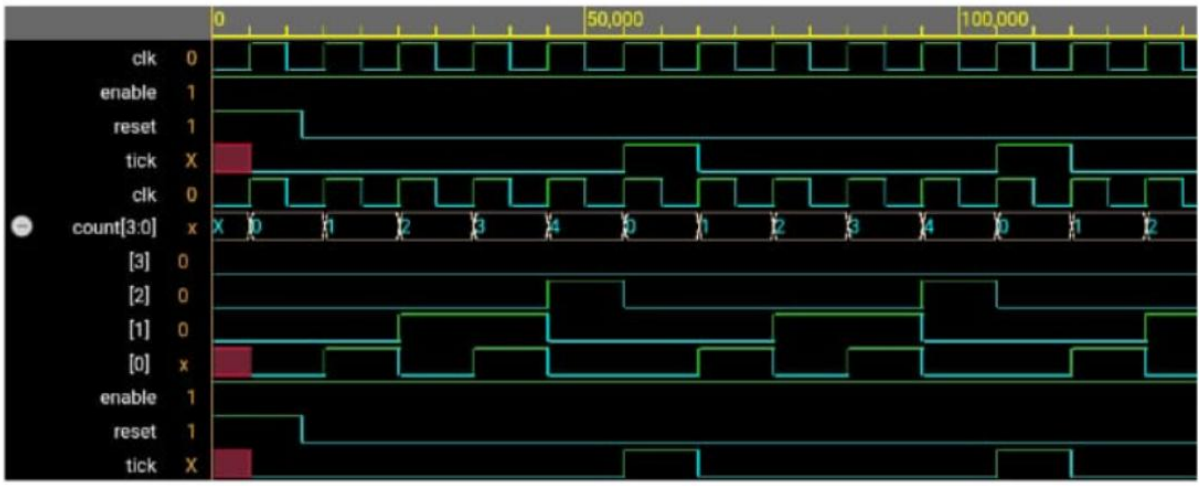


EXAMPLE:

: N = 4. The done signal becomes 1 whenever the counter reaches N−1, then returns to 0 on the next clock edge.

Clock event	Reset (rst)	Counter (count)	done
Reset active	1	0	0
First rising edge after reset	0	1	0
Second rising edge	0	2	0
Third rising edge	0	3	0
Fourth rising edge	0	0	1
Fifth rising edge	0	1	0
Sixth rising edge	0	2	0
Seventh rising edge	0	3	0
Eighth rising edge	0	0	1

OUTPUT WAVEFORM:



6. Ring Counter

WORKING:

A ring counter is a shift-register counter in which the last stage feeds back to the first stage. It is built from flip-flops that update together on each active clock edge. Reset loads a one-hot initial state, such as 0001. Each clock shifts the single logic one to the next stage and eventually circulates it back to the beginning. The waveform demonstrates the repeating state sequence and the effect of reset and enable.

VERILOG CODE:

```
// Code your design here
module ring_counter4(
    input clk, reset,
enable,
    output reg [3:0] q
);
always @(posedge clk or
posedge reset) begin
    if (reset)
        q <= 4'b0001;
    else if (enable)
        q <=
{q[2:0],q[3]};
end
endmodule
```

TESTBENCH:

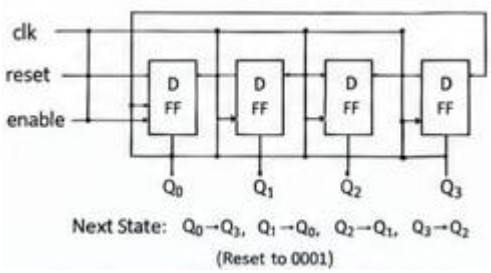
```
// Code your testbench
here
// or browse Examples
`timescale 1ns/1ps
module tb;
reg clk=0, reset=1,
enable=1;
wire [3:0] q;

always #5 clk=~clk;

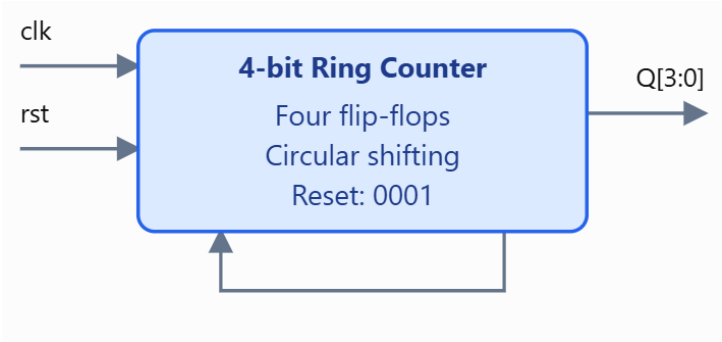
ring_counter4
dut(clk,reset,enable,q);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);
    #12 reset=0;
    #100;
    $finish;
end
endmodule
```

CIRCUIT DESIGN:

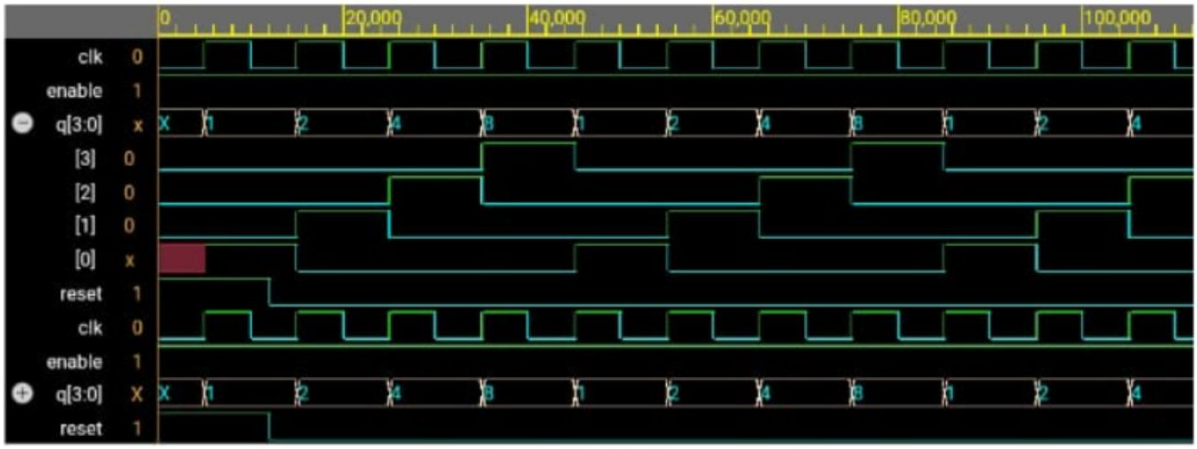


BLOCK DIAGRAM:



Clock edge	Q[3:0]
Reset	0001
1	0010
2	0100
3	1000
4	0001
5	0010
6	0100

OUTPUT WAVEFORM:



7. Johnson Counter

WORKING:

A Johnson counter is a shift-register counter that feeds the inverted output of the last stage back to the first stage. On each active clock edge, the flip-flops shift their values and the inverted feedback enters the first stage. For a four-stage implementation, the counter cycles through a sequence of distinct states before repeating. Reset places the counter in its initial state. The testbench shows the state progression and confirms the repeating sequence.

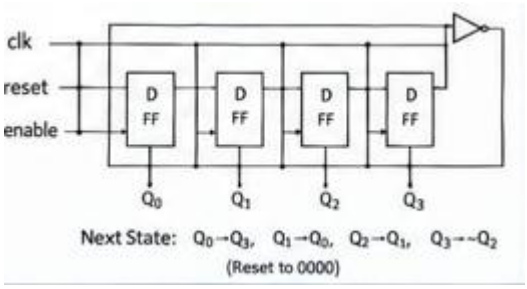
VERILOG CODE:

```
1  // Code your design here
2  module johnson_counter4(
3      input clk, reset,
   enable,
4      output reg [3:0] q
5  );
6  always @(posedge clk or
   posedge reset) begin
7      if (reset)
8          q <= 4'b0000;
9      else if (enable)
10         q <=
   {q[2:0],~q[3]};
11 end
12 endmodule
```

TESTBENCH:

```
1  // Code your testbench
   here
2  // or browse Examples
3  `timescale 1ns/1ps
4  module tb;
5  reg clk=0, reset=1,
   enable=1;
6  wire [3:0] q;
7
8  always #5 clk=~clk;
9
10 johnson_counter4
   dut(clk,reset,enable,q);
11
12 initial begin
13     $dumpfile("dump.vcd");
14     $dumpvars(0,tb);
15     #12 reset=0;
16     #120;
17     $finish;
18 end
19 endmodule
```

CIRCUIT DESIGN:



BLOCK DIAGRAM:

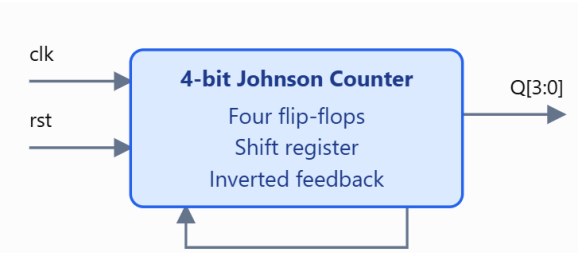
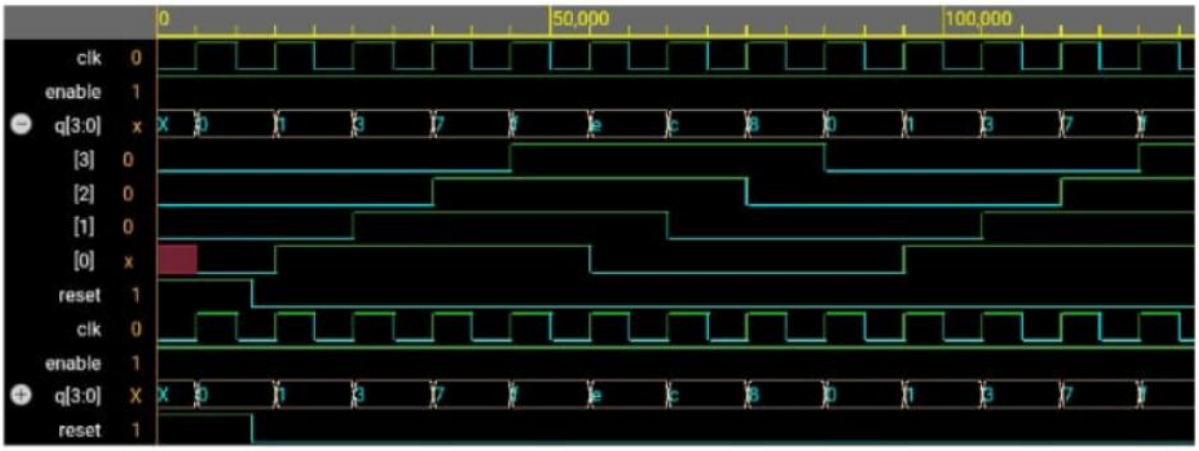


Figure 7: Block diagram of a 4-bit Johnson counter

Clock edge	Q[3:0]
Reset	0000
1	0001
2	0011
3	0111
4	1111
5	1110
6	1100
7	1000
8	0000

OUTPUT WAVEFORM:



8. Synchronous Counter

WORKING:

A synchronous counter uses a common clock signal for all flip-flops, so all state bits update together. The counter increments its binary value on each enabled active clock edge. Reset initializes the count, and enable allows the sequence to pause or continue. For a 4-bit counter, the sequence runs from 0000 through 1111 and wraps around modulo 16. The waveform illustrates synchronized state transitions and the effect of control inputs.

VERILOG CODE:

```
// Code your design here
module sync_counter4(
    input clk, reset,
enable,
    output reg [3:0] q
);
always @(posedge clk or
posedge reset) begin
    if (reset)
        q <= 0;
    else if (enable)
        q <= q + 1'b1;
end
endmodule
```

TESTBENCH:

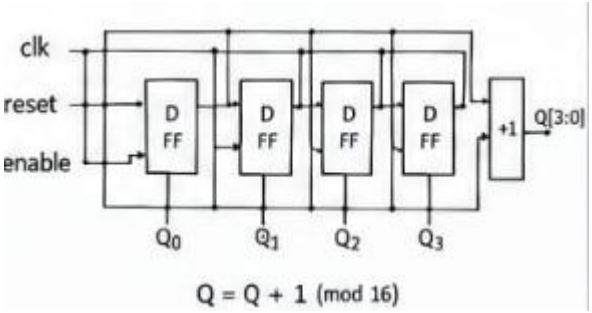
```
// Code your testbench
here
// or browse Examples
`timescale 1ns/1ps
module tb;
reg clk=0, reset=1,
enable=1;
wire [3:0] q;

always #5 clk=~clk;

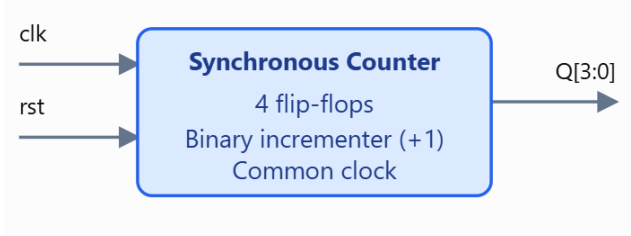
sync_counter4
dut(clk,reset,enable,q);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);
    #12 reset=0;
    #180;
    $finish;
end
endmodule
```

CIRCUIT DESIGN:

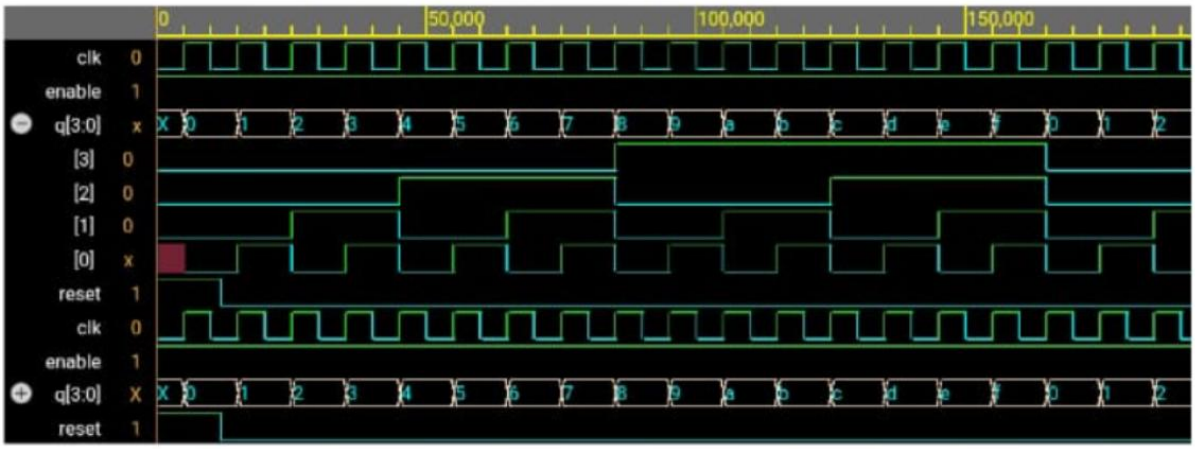


BLOCK DIAGRAM:



Clock edge	Output Q
Reset	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000

OUTPUT WAVEFORM:



9. Asynchronous Counter

WORKING:

An asynchronous counter, also called a ripple counter, clocks the first flip-flop from the external clock and uses preceding stages to trigger later stages. Because transitions ripple through the stages, the output bits do not change at exactly the same instant. The counter progresses through binary states as the clock toggles. Reset initializes all stages and enable controls operation according to the implementation. The waveform displays the resulting count sequence and ripple-style transitions.

VERILOG CODE:

```
// Code your design here
module async_counter4(
    input clk, reset,
enable,
    output reg [3:0] q
);
always @(posedge clk or
posedge reset) begin
    if (reset) q[0] <= 0;
    else if (enable) q[0]
<= ~q[0];
end

always @(negedge q[0] or
posedge reset) begin
    if (reset) q[1] <= 0;
    else q[1] <= ~q[1];
end

always @(negedge q[1] or
posedge reset) begin
    if (reset) q[2] <= 0;
    else q[2] <= ~q[2];
end

always @(negedge q[2] or
posedge reset) begin
    if (reset) q[3] <= 0;
    else q[3] <= ~q[3];
end
endmodule
```

TESTBENCH:

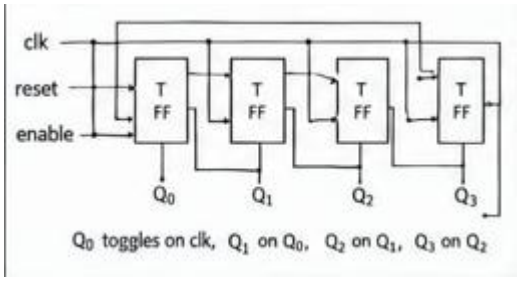
```
// Code your testbench
here
// or browse Examples
`timescale 1ns/1ps
module tb;
reg clk=0, reset=1,
enable=1;
wire [3:0] q;

always #5 clk=~clk;

async_counter4
dut(clk,reset,enable,q);

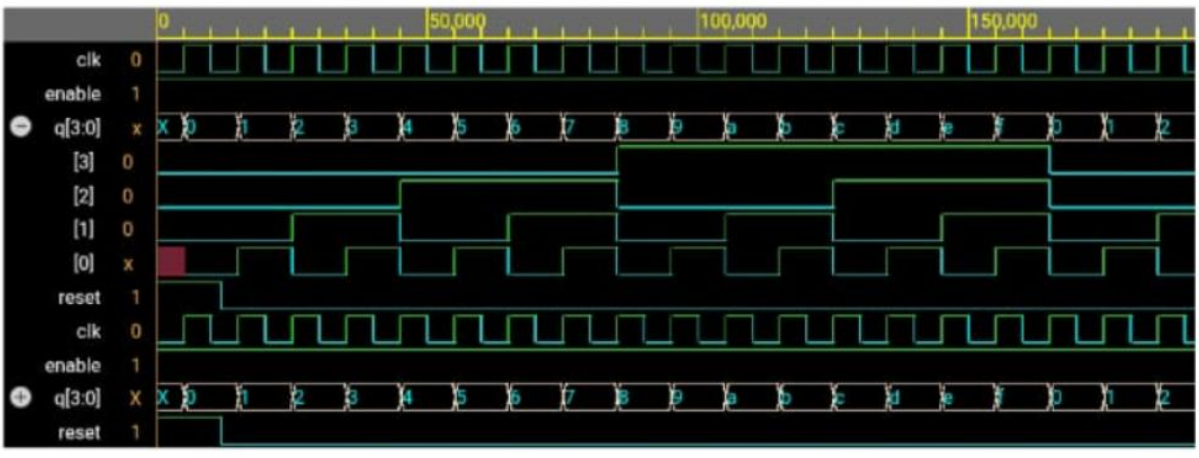
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);
    #12 reset=0;
    #180;
    $finish;
end
endmodule
```

CIRCUIT DESIGN:



Clock pulse	Q[3]	Q[2]	Q[1]	Q[0]	Output Q
Reset	0	0	0	0	0000
1	0	0	0	1	0001
2	0	0	1	0	0010
3	0	0	1	1	0011
4	0	1	0	0	0100
5	0	1	0	1	0101
6	0	1	1	0	0110
7	0	1	1	1	0111
8	1	0	0	0	1000
9	1	0	0	1	1001
10	1	0	1	0	1010

OUTPUT WAVEFORM:



10. Shift Register

WORKING:

A shift register stores binary data in a series of flip-flops and moves the data by one position on each active clock edge. Serial input supplies new data to the first stage, while each stage passes its stored bit to the next stage. Reset initializes the register and enable determines whether shifting occurs. The parallel outputs show the current contents of the register, and a serial output can expose the shifted-out bit. The testbench applies serial data and verifies the shifting sequence.

VERILOG CODE:

```
// Code your design here
module shift_register4(
    input clk, reset,
enable, serial_in,
    output reg [3:0] q,
    output serial_out
);
assign serial_out = q[3];

always @(posedge clk or
posedge reset) begin
    if (reset)
        q <= 0;
    else if (enable)
        q <=
{q[2:0],serial_in};
end
endmodule
```

TESTBENCH:

```
// Code your testbench
here
// or browse Examples
`timescale 1ns/1ps
module tb;
reg clk=0, reset=1,
enable=1, serial_in=0;
wire [3:0] q;
wire serial_out;

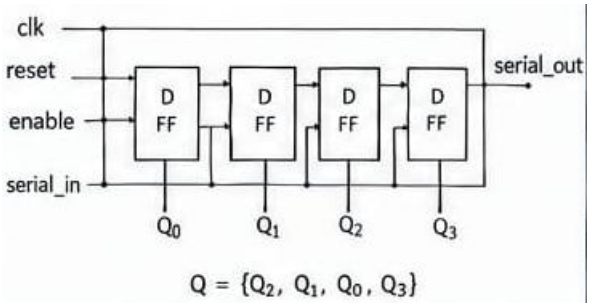
always #5 clk=~clk;

shift_register4 dut(

clk,reset,enable,serial_in
,q,serial_out
);

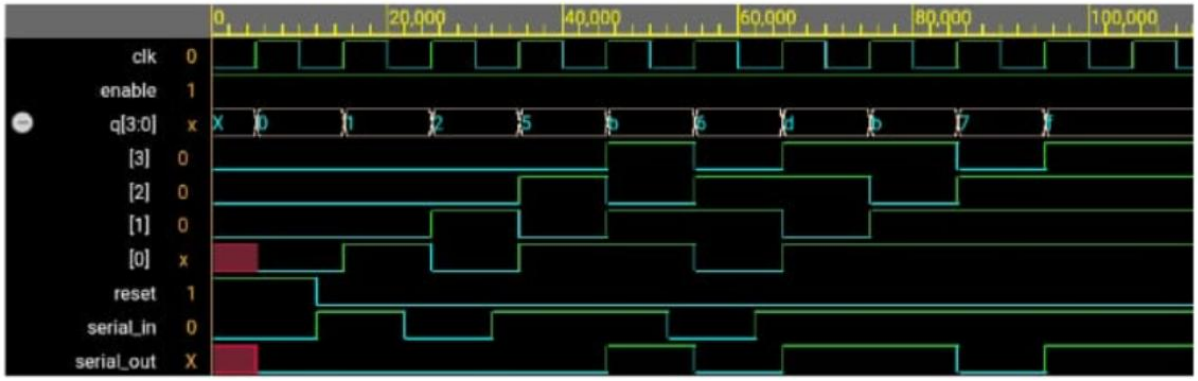
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);
    #12 reset=0;
    serial_in=1; #10;
    serial_in=0; #10;
    serial_in=1; #10;
    serial_in=1; #10;
    serial_in=0; #10;
    serial_in=1; #10;
    #40;
    $finish;
end
endmodule
```

CIRCUIT DESIGN:



Clock pulse	Serial input (SI)	Output Q
Reset	0	0000
1	1	0001
2	0	0010
3	1	0101
4	1	1011
5	0	0110

OUTPUT WAVEFORM:



11. Divide by N-counter

WORKING:

A divide-by-N counter generates a slower timing signal from a faster input clock by counting a fixed number of clock events. The counter advances while enabled and is cleared or reloaded when the selected terminal count is reached. Terminal-count logic then toggles or pulses the divided output. The division ratio depends on the counter threshold and output-toggle scheme used in the Verilog design. The waveform verifies the repeated count cycle and the resulting divided-clock behavior.

VERILOG CODE:

```
// Code your design here
module divide_by_n #(
    parameter N=5,
    parameter WIDTH=3
)(
    input clk, reset,
    output reg clk_div
);
reg [WIDTH-1:0] count;

always @(posedge clk or
posedge reset) begin
    if (reset) begin
        count <= 0;
        clk_div <= 0;
    end
    else if (count == N-
1) begin
        count <= 0;
        clk_div <=
~clk_div;
    end
    else
        count <= count +
1'b1;
end
endmodule
```

TESTBENCH:

```
// Code your testbench
here
// or browse Examples
`timescale 1ns/1ps
module tb;
reg clk=0, reset=1;
wire clk_div;

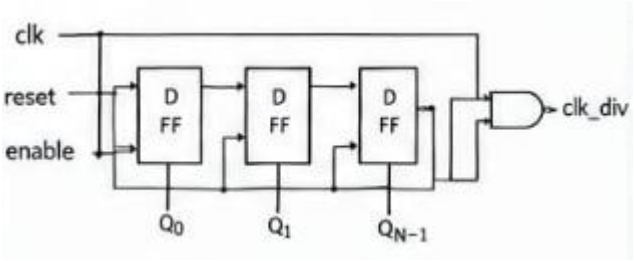
always #5 clk=~clk;

divide_by_n #
(.N(5),.WIDTH(3))

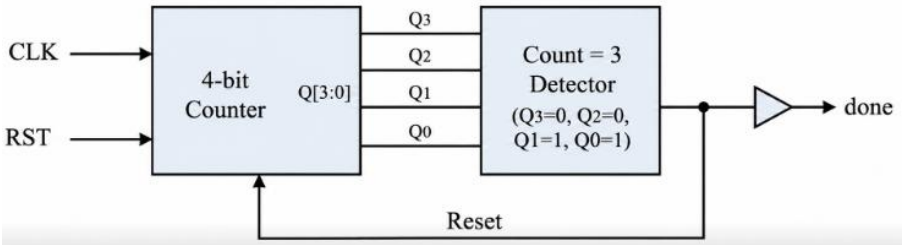
dut(clk,reset,clk_div);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);
    #12 reset=0;
    #200;
    $finish;
end
endmodule
```

CIRCUIT DESIGN:

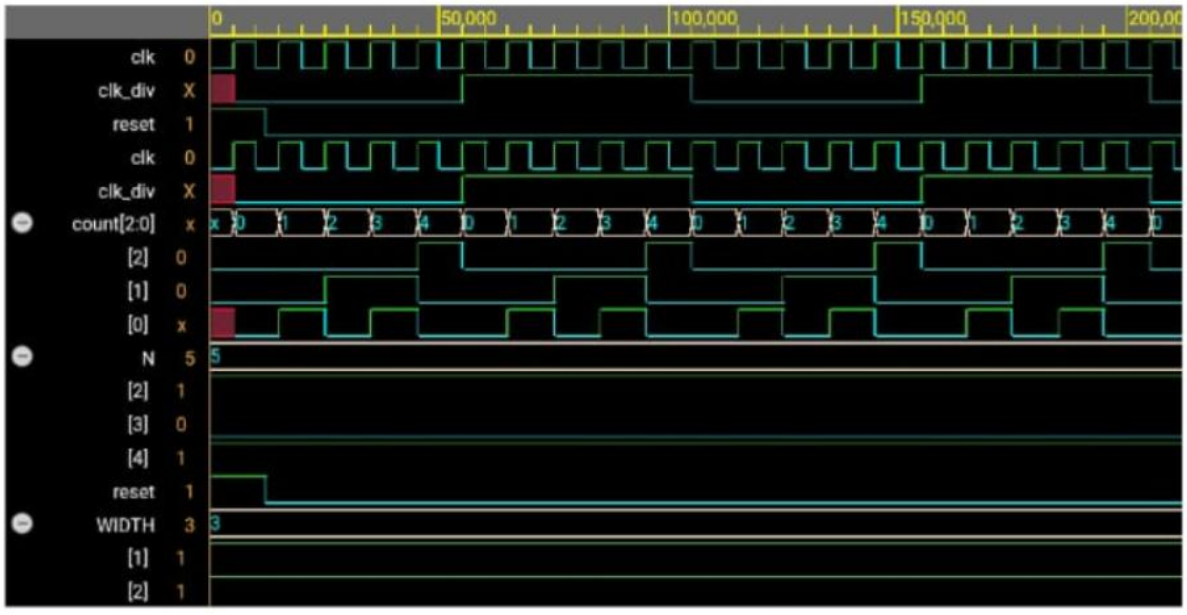


BLOCK DIAGRAM:



Clock pulse	Count before edge	Count after edge	done
Reset	—	0	0
1	0	1	0
2	1	2	0
3	2	3	0
4	3	0	1
5	0	1	0
6	1	2	0
7	2	3	0
8	3	0	1

OUTPUT WAVEFORM:



12. Mealy FSM

WORKING:

A Mealy finite state machine produces outputs based on both its current state and current input. State is stored in flip-flops and updated on the active clock edge, while combinational logic determines the next state and output. In a sequence detector, the states represent how much of the target pattern has been observed. The machine asserts the detection output when the required sequence is completed, potentially in the same cycle as the final input bit. The testbench applies input sequences to verify detection and state transitions.

VERILOG CODE:

```
// Code your design here
module mealy_detector101(
    input clk, reset,
din,
    output reg detected
);
reg [1:0] state;
parameter S0=0, S1=1,
S10=2;

always @(posedge clk or
posedge reset) begin
    if (reset) begin
        state <= S0;
        detected <= 0;
    end
    else begin
        detected <= 0;
        case(state)
            S0:
                if (din)
state <= S1;
                else
state <= S0;

            S1:
                if (din)
state <= S1;
                else
state <= S10;

            S10:
                if (din)
begin
                    state
<= S1;

detected <= 1;
                end
                else
state <= S0;

            default:
state <= S0;
        endcase
    end
end
endmodule
```

TESTBENCH:

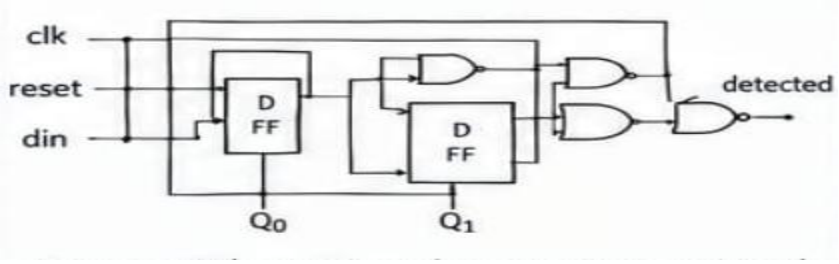
```
// Code your testbench
here
// or browse Examples
`timescale 1ns/1ps
module tb;
reg clk=0, reset=1, din=0;
wire detected;

always #5 clk=~clk;

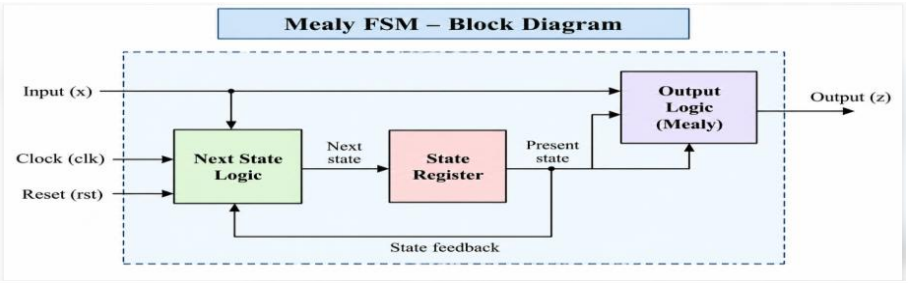
mealy_detector101
dut(clk,reset,din,detected
);

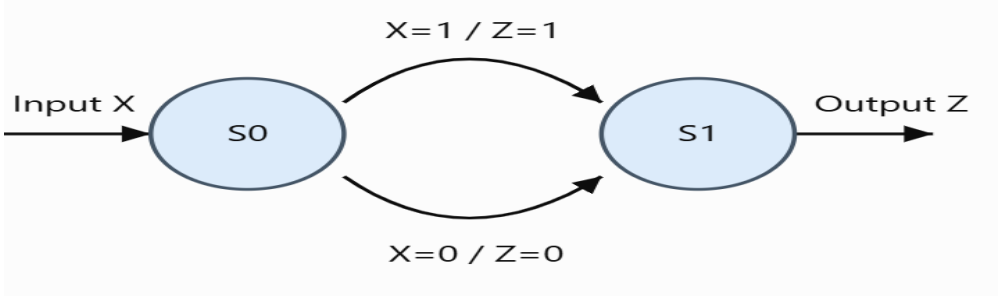
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);
    #12 reset=0;
    din=1; #10;
    din=0; #10;
    din=1; #10;
    din=1; #10;
    din=0; #10;
    din=1; #10;
    #30;
    $finish;
end
endmodule
```

CIRCUIT DIAGRAM:



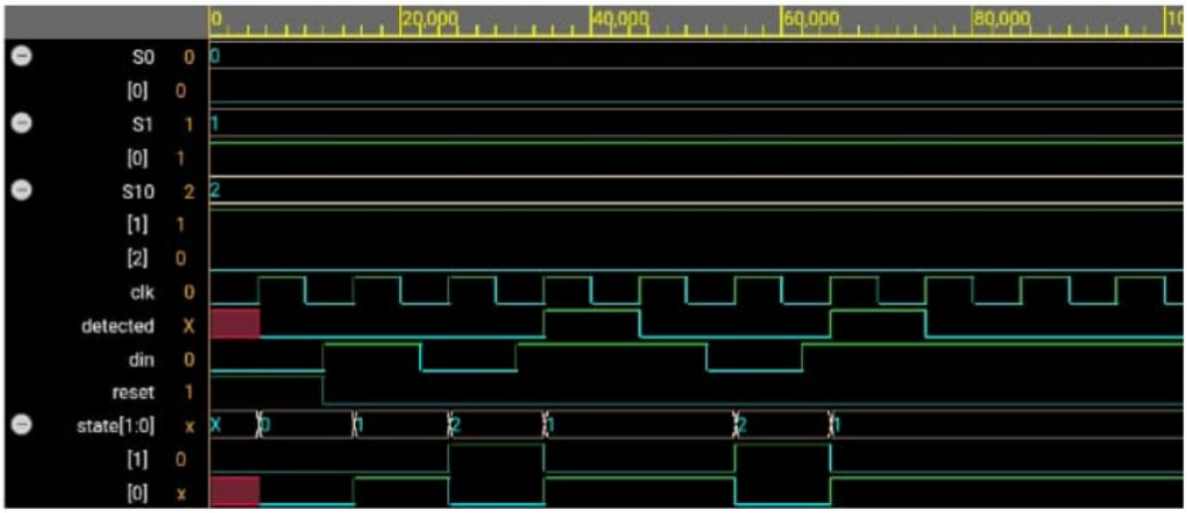
BLOCK DIAGRAM:





Clock cycle	Input X	Output Z
Reset	0	0
1	1	1
2	0	0
3	1	1
4	1	1
5	0	0

OUTPUT WAVEFORM:



13. Moore FSM

WORKING:

A Moore finite state machine produces outputs based only on its current state. Clocked flip-flops store the state, and combinational logic determines the next state from the current state and input. For a sequence detector, each state represents progress toward recognizing the target pattern. The detection output is asserted when the machine enters the designated detection state, so it depends on state rather than directly on the current input. The waveform verifies state transitions and output timing for the supplied input sequence.

VERILOG CODE:

```
// Code your design here
module moore_detector101(
    input clk, reset,
din,
    output reg detected
);
reg [1:0] state;
parameter S0=0, S1=1,
S10=2, FOUND=3;

always @(posedge clk or
posedge reset) begin
    if (reset)
        state <= S0;
    else begin
        case(state)
            S0:
                if (din)
state <= S1;
                else
state <= S0;

            S1:
                if (din)
state <= S1;
                else
state <= S10;

            S10:
                if (din)
state <= FOUND;
                else
state <= S0;

            FOUND:
                if (din)
state <= S1;
                else
state <= S10;

            default:
state <= S0;
        endcase
    end
end

always @(*) begin
    detected = (state ==
FOUND);
end
endmodule
```

TESTBENCH:

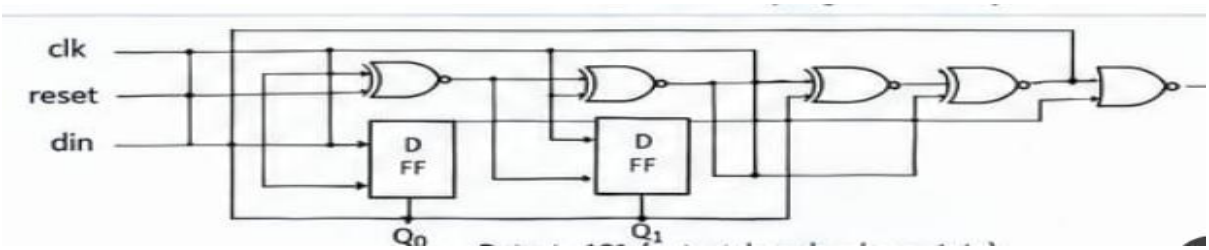
```
// Code your testbench
here
// or browse Examples
`timescale 1ns/1ps
module tb;
reg clk=0, reset=1, din=0;
wire detected;

always #5 clk=~clk;

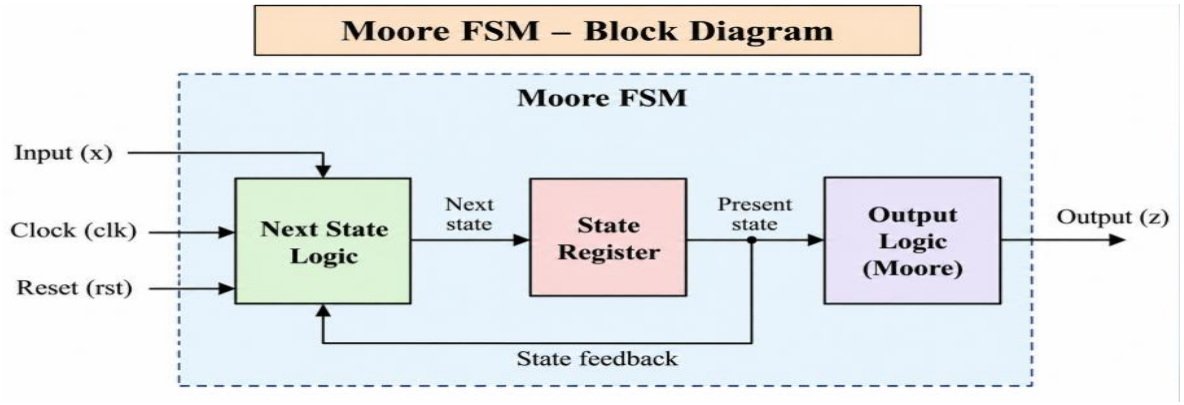
moore_detector101
dut(clk,reset,din,detected
);

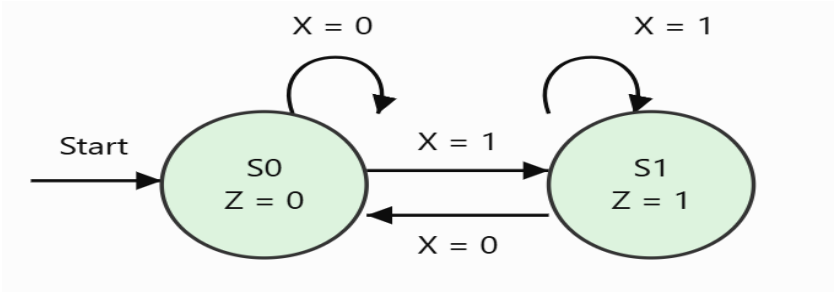
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb);
    #12 reset=0;
    din=1; #10;
    din=0; #10;
    din=1; #10;
    din=1; #10;
    din=0; #10;
    din=1; #10;
    #30;
    $finish;
end
endmodule
```

CIRCUIT DESIGN:



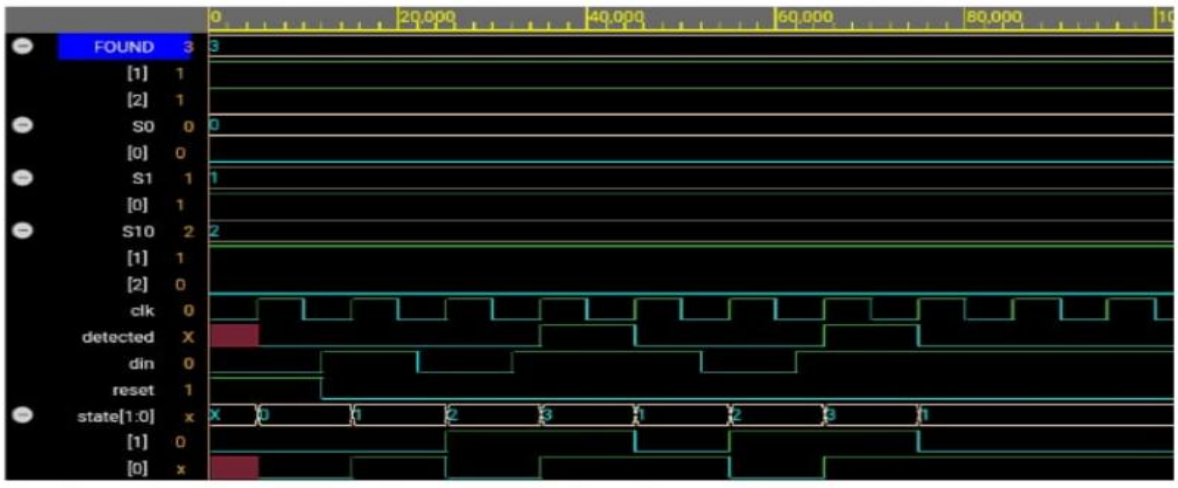
BLOCK DIAGRAM:





Clock cycle	Input X	Present state	Output Z
Reset	0	S0	0
1	1	S0	0
2	0	S1	1
3	1	S0	0
4	1	S1	1
5	0	S1	1

OUTPUT WAVEFORM:



14.Random Sequence Generator (LFSR)

WORKING:

A linear-feedback shift register (LFSR) can generate a deterministic pseudo-random bit sequence using shift registers and XOR/XNOR feedback. On each active clock edge, the stored bits shift by one position and a feedback bit is calculated from selected tap positions. Reset loads a nonzero starting state, and enable controls whether the sequence advances. The sequence repeats after a finite number of states determined by the register width and feedback polynomial. The uploaded circuit diagram is included; the waveform placeholder remains because a waveform screenshot was not present in the supplied files.

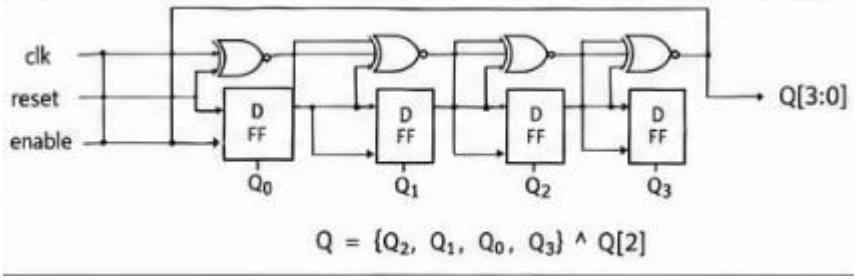
VERILOG CODE:

```
1  // Code your design here
2  module lfsr4(
3      input clk, reset,
   enable,
4      output reg [3:0] q
5  );
6  always @(posedge clk or
   posedge reset) begin
7      if (reset)
8          q <= 4'b0001;
9      else if (enable)
10         q <= {q[2:0],q[3]
   ^ q[2]};
11 end
12 endmodule
```

TESTBENCH:

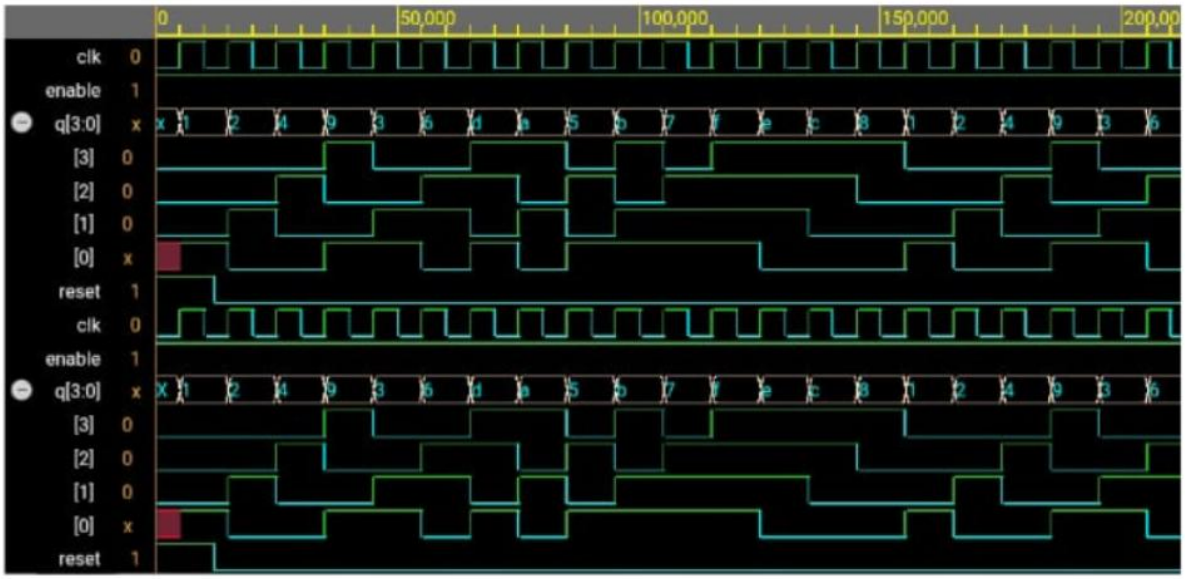
```
1  // Code your testbench
   here
2  // or browse Examples
3  `timescale 1ns/1ps
4  module tb;
5  reg clk=0, reset=1,
   enable=1;
6  wire [3:0] q;
7
8  always #5 clk=~clk;
9
10 lfsr4
   dut(clk,reset,enable,q);
11
12 initial begin
13     $dumpfile("dump.vcd");
14     $dumpvars(0,tb);
15     #12 reset=0;
16     #200;
17     $finish;
18 end
19 endmodule
```

CIRCUIT DESIGN:



Clock pulse	Output Q
Reset	0001
1	0011
2	0111
3	1111
4	1110
5	1101
6	1010
7	0101
8	1011

OUTPUT WAVEFORM:



CONCLUSION

Thus, the various digital logic circuits were successfully designed and implemented using Verilog HDL. The corresponding testbenches were developed to verify the functionality of each design through simulation. The obtained waveforms and simulation results confirmed the expected behaviour of the circuits. This work provided practical knowledge of digital system design, Verilog coding, functional verification, and waveform analysis, strengthening the understanding of digital electronics and hardware description languages.